

How early can bus bunching be predicted, and is that early enough to act on?

A horizon-resolved machine learning study of eight high-frequency SL bus lines, Stockholm 2024

Bruno Rinaldo and [partner name]

AH2179 Applied AI in Transportation, KTH Royal Institute of Technology. Project Option 1 (Public Transport). Code and full analysis: [\[GitHub repository link\]](#)

Abstract. Bus bunching is a self-reinforcing instability of high-frequency transit, and detecting it after the fact is trivial; the operational question is how much warning a model can give. Using 12.7 million GTFS-RT trip-stop observations from eight high-frequency SL bus lines over all of 2024, joined to ERA5 weather and service alerts, we frame bunching prediction as a horizon-resolved binary classification problem at $k = 1, 2, 3, 5$ and 8 stops ahead, and compare a persistence baseline, logistic regression, gradient boosting and a 1D convolutional sequence model on a strictly temporal split. Gradient boosting reaches 0.81 PR-AUC three stops ahead against a 2.4 % base rate, catching 75 % of events at 80 % precision, but only 20 % at eight stops. The usable warning window is therefore about one to three minutes, which supports a controller decision aid rather than automated control. Diagnostics reject the project's own dwell-growth hypothesis, find the model's weakness to be one specific line rather than any weather or time-of-day condition, and show clean transfer to unseen lines within the trunk-bus regime.

1. Introduction and problem formulation

Bus bunching is not a random service failure but a structural instability of high-frequency transit. A vehicle that falls slightly behind schedule arrives at the next stop to a larger accumulated crowd, dwells longer to board it, and falls further behind. The vehicle behind it then reaches stops that were recently cleared, dwells less, and closes the gap. Headway variance therefore amplifies along a route rather than dissipating, and the equilibrium of an uncontrolled high-frequency line is pairs of buses travelling together separated by long gaps (Newell and Potts, 1964; Daganzo, 2009). Dwell time, not lateness as such, is the variable that drives the loop.

The passenger cost is carried almost entirely by waiting time. For random passenger arrivals at a stop, expected wait is $E[W] = (E[h]/2)(1 + CV^2)$, where CV is the coefficient of variation of the observed headway. Irregularity therefore inflates average wait quadratically in CV even when the timetabled frequency is unchanged, which is why an operator can run every scheduled departure and still deliver a visibly worse service.

Detecting bunching once it has happened is close to trivial: the headway is already short, and any threshold rule will find it. The operationally interesting question is whether it can be seen coming, and with how much warning. Control actions available to an operator (holding a bus at a stop, short-turning, skip-stopping, or simply directing a controller to watch a specific vehicle pair) all need lead time. A prediction that arrives one minute ahead of a bunching event is a different product from one that arrives eight minutes ahead, and the difference is not cosmetic: it decides whether the system can be an automated trigger, a controller decision aid, or nothing at all.

Research question. How early can bus bunching be predicted from real-time operational data alone, and is that lead time long enough to be operationally actionable? This is decomposed into four sub-questions: **RQ1 (descriptive)** where and when does headway irregularity form, and does it grow along a route; **RQ2 (mechanism)** does dwell-time growth predict downstream bunching better than arrival delay does; **RQ3 (core)** how does predictive performance decay as the prediction horizon k increases; and **RQ4 (generalisation)** does a model trained on one set of lines transfer to another.

Hypotheses. H1: headway CV increases with position along the route. H2: dwell-growth features outperform delay-only features at horizons k of 3 or more. H3: performance decays with k , with usable performance up to roughly $k = 3$ to 5 stops. H4: adverse weather raises the base rate of bunching but does not materially change which features are predictive. All four are tested explicitly below, and two of them do not survive contact with the data.

The methodological contribution is to treat the prediction horizon as the experimental variable rather than a fixed design choice. Rather than reporting one accuracy figure for one arbitrary lead time, every model is trained and evaluated at five horizons ($k = 1, 2, 3, 5, 8$ stops, converted to minutes through the median inter-stop running time of 63 s) and benchmarked against a persistence baseline that represents what an operator already knows without any model at all.

2. Data and preparation

Sources. Twelve monthly GTFS-RT files for 2024 (roughly 18.7 million trip-stop observations per month across all modes, of which 88.1 % are bus), joined to static GTFS stops, routes, trips and transfers, plus a service-alert file of 1,972,779 alert-entity rows. Hourly weather was taken from the Open-Meteo ERA5 reanalysis archive (Hersbach et al., 2020) rather than the source named in the brief, because it is free, keyless, gapless for the full year, and gridded at a resolution that matches the geographic spread of a bus line. Stops were snapped to a 0.1 degree grid, yielding 9 grid points covering all 380 stops used here and exactly $9 \times 366 \times 24 = 79,056$ weather rows, confirming a complete series.

Line selection is itself a result. Candidate lines were first filtered on median scheduled peak headway of 600 s or less, and the eight highest-volume survivors selected. A full-year completeness scan then showed that the missing rate of observed arrival times is very far from uniform across lines: two of the provisional eight, lines 1 and 3, were missing 44.15 % and 63.90 % of their real-time arrivals respectively, against 1 % to 12 % for the rest. Headways built on those lines would have been mostly interpolation, so both were dropped and replaced by the next best candidates that still met the frequency bar. The final set is lines 4, 116, 117, 179, 401, 474, 541 and 607: 12,683,862 trip-stop rows for the full year, all high-frequency trunk services. This is a scope boundary worth stating early, because it bounds what RQ4 can claim.

Three data traps that changed the pipeline. First, the `_sfm` time fields exceed 86,400 s for post-midnight service in about 0.6 % of rows, so timestamps were reconstructed by interval addition to service midnight rather than modulo arithmetic, which would have produced large negative headways at exactly the hours when service is thinnest. Second, `route_id` is not a stable line identifier: about 2.6 % of bus rows carry a route-variant suffix absent from `routes.txt`, so grouping on it silently fragments one physical line into disconnected leader-follower chains at precisely the stops where variants converge and overtaking is most likely. Leader-follower pairs were therefore grouped on `(date, LineNumber, direction_id, observed_stop_id)`, which also matches what a waiting passenger experiences. Third, the two documented merge-quality flags are false for 100 % of rows in the final dataset: they flag a static-to-realtime join failure, not a missing real-time update, and are not usable as the data-quality filter the project plan assumed they would be.

Headway and the bunching label. The leader of a trip at a stop is defined as the trip scheduled to arrive immediately before it there, which keeps leader identity fixed along the whole trip (an observed-order definition would reassign it at every overtake, making any multi-stop trajectory feature meaningless). With `headway_ratio = observed headway / scheduled headway`, a stop is labelled bunching when $|\text{headway_ratio}| < 0.25$. Two label corrections made during construction changed reported rates materially and are reported rather than hidden. (i) Unbounded observed headways let corrupted pairings (reaching -47 scheduled headways) enter the aggregate; observed headway is now bounded to `[-1800, 10800]` s, which keeps the physically plausible negative values (93 % of negatives lie within 30 minutes of zero and represent genuine overtaking) while discarding the implausible tail. (ii) The test must use the absolute ratio. A signed test labels a follower that has overtaken its leader by 25 minutes as bunching, which is the opposite of the truth: bunching is physical proximity, and proximity is symmetric about zero. Together these corrections reduced the measured base rate from an inflated 2.91 % to 2.29 %, so every figure in this report is the conservative version.

Targets and leakage control. For horizon `k`, `y_k = 1` if the same fixed pair is bunching at stop `n+k` of the same trip, computed as `lead(is_bunched, k)` over `(date, trip_id)` ordered by `stop_sequence` and gated on the headway at `n+k` being valid. Trips that do not reach stop `n+k` return null rather than zero, and a companion `target_valid_k` flag makes that distinguishable; treating those rows as negatives would bias every metric downward. Leakage is prevented by construction rather than by inspection: every feature expression uses only `lag()` and backward window frames, and no feature expression calls `lead()` at all. Valid rows fall from 12.12 M at `k = 1` to 8.21 M at `k = 8`, while the positive rate rises slightly from 2.00 % to 2.33 %, consistent with variance accumulating along the route.

Known incompleteness. 13.1 % of rows have a null observed headway, inherited from the 8 to 10 % per-row missing real-time match (a null propagates to the following row's difference as well). Weather is null for 7.9 % of rows, exactly the rows whose observed timestamp is null, which confirms it is the same missingness rather than a second join failure. Dwell is null for 16.0 % of rows, mostly by design: dwell at the first and last stop of a trip is layover, not boarding. Missing-observation rates are also

volatile within the year, from under 1 % in good stretches to full-day outages, which is a caution against reading any month-to-month comparison as purely operational.

3. Descriptive analysis

Bunching affects 2.29 % of valid trip-stop observations across the eight lines at the $|\text{ratio}| < 0.25$ definition, and the label is not knife-edge sensitive: the rate moves from 1.82 % at a 0.20 threshold to 5.35 % at 0.50, with the primary threshold sitting on a smooth part of that curve. The rate is far from uniform across lines, and the same ordering appears again in the excess waiting time each line imposes on its passengers (Table 1).

Line	% bunched	Mean headway (s)	CV	Excess wait (min/pax)	Test PR-AUC, HGB k=3
4	4.10	573	0.49	1.24	0.846
179	3.06	674	0.58	1.97	0.777
117	2.30	887	0.59	2.63	0.804
607	1.96	861	0.53	2.03	0.877
474	1.88	706	0.52	1.64	0.628
541	1.53	844	0.38	1.04	0.797
401	1.34	914	0.43	1.48	0.809
116	0.87	1061	0.51	2.34	0.922

Table 1. Per-line bunching rate, headway regularity, excess waiting time attributable to irregularity, and (from Section 6) how well the gradient boosting model predicts each line. Excess wait is the difference between observed $E[W] = (E[h]/2)(1 + \text{CV squared})$ and the $\text{CV} = 0$ timetable benchmark.

H1: irregularity grows along the route, but not on every line. Lines 4 and 179 show the clean monotonic rise the hypothesis predicts (line 4 from CV about 0.40 near the start to about 0.55 at the end). Lines 116 and 607 are essentially flat along their length, and line 117 shows a step down after a mid-route gap caused by a short-turn pattern most trips do not traverse. An apparent sharp fall at the terminus of line 401 is a sample-size artifact: its stop 23 is observed 1,231 times in the year against roughly 45,000 at every earlier stop. Pooling all eight lines gives a mild upward drift (0.52 to about 0.56), but that pooled trend is carried by two lines rather than holding universally. H1 is supported in direction but not as a line-independent law, and a per-line view is necessary to see that.

Bunching rate by hour of day and line

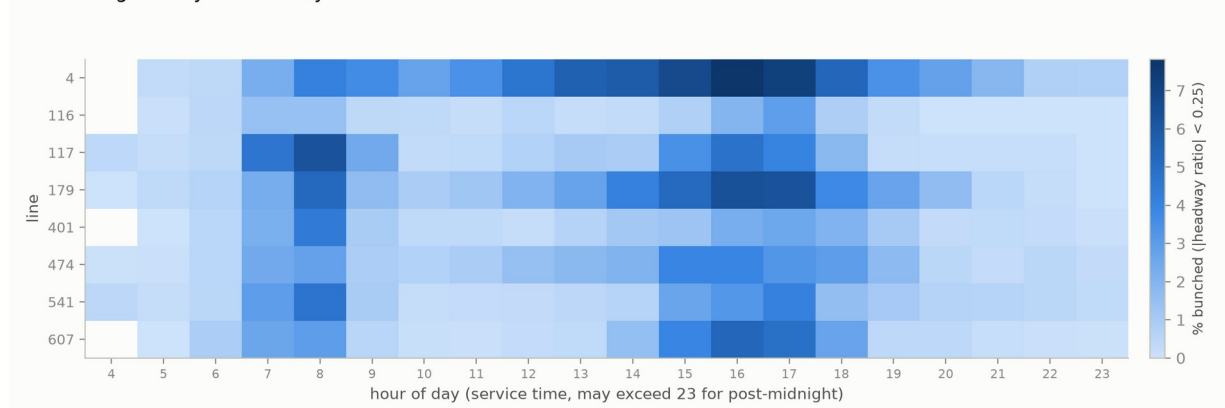


Figure 1. Bunching rate by hour of day and line. Rush-hour concentration is near-universal, but line 4 is elevated at almost every hour, not only at the peaks.

When and where. Bunching concentrates at the peaks, with lines 117, 179, 401 and 541 peaking at 07:00 to 08:00 and lines 4, 179, 474, 541 and 607 in the afternoon peak. Line 4 is the outlier of the set: highest at nearly every hour, above 7.5 % at 16:00, more than double most other lines at their own peak. The spatial view agrees rather than adding independent evidence: the eight worst stops in the network (Garnisonen 8.04 %, Banérgatan 7.70 %, Skanstull 7.44 %, Värtavägen 7.25 %, Eriksdal 7.21 %, Musikhögskolan 7.08 %, Stadion 6.86 %, Östra station 6.53 %) all lie on line 4's central corridor and carry nearly identical observation counts. Peak service is also less predictable in both directions, not merely busier: the peak-hour headway ratio distribution is flatter and fatter-tailed than off-peak on both sides of the mode, so a peak-tuned control policy has to manage gaps as much as bunches.

A seasonal trap. January is the worst month (3.38 %) and July by far the best (0.65 %), which invites a weather narrative. It is mostly an artifact of the timetable: July has the loosest scheduled headways of the year on these lines (median 900 s against 600 to 650 s in January to May) and among the fewest distinct trips. A wider scheduled denominator mechanically shrinks the ratio-based bunching rate at unchanged absolute adherence. Combined with the volatile feed-completeness noted in Section 2, month-level comparisons are the weakest evidence in this dataset and are not used to test H4; Section 6 tests it on the model side instead, where the confound does not apply.

The mechanism is cumulative, not instantaneous. Interchange stops have a mean dwell of 39.4 s (median 21 s) against 16.3 s (median 12 s) at ordinary stops, so the stop attribute most closely tied to boarding load does behave as the mechanism predicts. But the single-stop correlation between a trip's own delay growth and its own headway ratio is only -0.035, and between the leader's delay growth and the follower's ratio only +0.018. Both are in the predicted direction and both are far too weak to carry the causal story on their own. The reading taken here is not that the mechanism is absent but that it does not appear at a one-stop resolution, which is the direct justification for the trajectory feature group (lagged headway ratios, closing speed, a three-stop rolling delay sum) and for testing H2 through model ablation rather than bivariate correlation.

4. Modelling approach

Features. 26 numeric, 6 boolean and 3 categorical inputs in eight groups: current state (headway ratio, arrival and departure delay, dwell, dwell relative to the stop median), trajectory (headway ratio at $n-1$ to $n-3$, closing speed over three stops, cumulative delay growth, lagged running speeds), leader (the scheduled predecessor's dwell, delay and delay growth at stop n), route position, temporal context, stop attributes (interchange flag, parent station, routes serving the stop), a binary active-alert flag, and weather. Weather enters as a feature without violating the leakage rule because it is exogenous and forecastable at prediction time. Temperature is entered continuously, with a quadratic term for the linear model only, rather than binned: the hypothesised U-shape (boarding slows in both cold and heat) needs a nonlinear term in a GLM, but hand-picked bin edges discard information at every boundary and the tree model never needed them. No unsupervised clustering is used; the groupings present (weather grid, weather-code classes) are domain discretisations, and adding a learned stop clustering without evidence that the raw stop attributes underperform would have been unjustified complexity.

Split. Temporal: January to September train (9,543,333 rows, 75.3 %), October validation (1,088,445, 8.6 %), November to December test (2,052,084, 16.2 %). A random split is not defensible here, because adjacent stops of the same trip are near-duplicates in feature space and would appear on both sides of the boundary. The split is assigned from the date column directly, never from a derived timestamp: an earlier version assigned it from the observed arrival time, and because that field is null for about 8 % of rows, those rows fell through to the test branch regardless of their actual date, putting January rows in a November to December test set. Section 6 quantifies what the choice of split is actually worth.

Class imbalance and metrics. The base rate is 2.0 to 2.4 % depending on horizon, so accuracy is meaningless and was not used. Models are trained with class weights, never resampled, and the test set's class balance is left untouched. PR-AUC is the primary metric, reported alongside the Brier score for calibration and recall at a fixed 80 % precision operating point, which is the number that actually matters to a control room deciding how many false alarms it will tolerate.

Models. Four, in increasing capacity. (1) A persistence baseline that predicts the pair's state at $n+k$ equals its state at n . This is not a strawman: it is exactly what a controller watching a live map already knows, so it is the standard a learned model has to beat to justify existing. (2) Logistic regression on the full engineered feature set, one-hot encoded and standardised, included for its interpretable coefficients rather than its accuracy. (3) HistGradientBoostingClassifier (200 iterations), chosen over a plain gradient booster specifically because it handles the 13 % null headway family and 7.9 % null weather natively, without imputation that would fabricate values at exactly the rows where the feed failed. (4) A Keras 1D-CNN over the six-step window $n-5$ to n of (headway ratio, dwell, delay growth) plus a fourth channel marking which timesteps were missing: two causal Conv1D layers (24 filters, kernel 3), global average pooling, a 16-unit dense layer with 0.2 dropout, sigmoid output, Adam at $1e-3$, batch 8192, early stopping on validation PR-AUC. This is also the .keras export the assignment requires, and it was trained and reported regardless of whether it won.

5. Results: how performance decays with lead time

k	minutes	persistence P / R	logreg PR-AUC	HGB PR-AUC	Keras PR-AUC	HGB recall @ 80 % prec.
1	1.05	0.89 / 0.84	0.586	0.898	0.917	0.95
2	2.10	0.84 / 0.75	0.563	0.860	0.855	0.86
3	3.15	0.81 / 0.67	0.539	0.814	0.786	0.75
5	5.25	0.75 / 0.55	0.493	0.696	0.682	0.47
8	8.40	0.67 / 0.41	0.425	0.623	0.562	0.20

Table 2. Test-set results (November to December 2024, all eight lines pooled, scored once). Persistence has no probability score, so PR-AUC is undefined for it and its single precision/recall operating point is given instead. Minutes use the median inter-stop running time of 63 s.

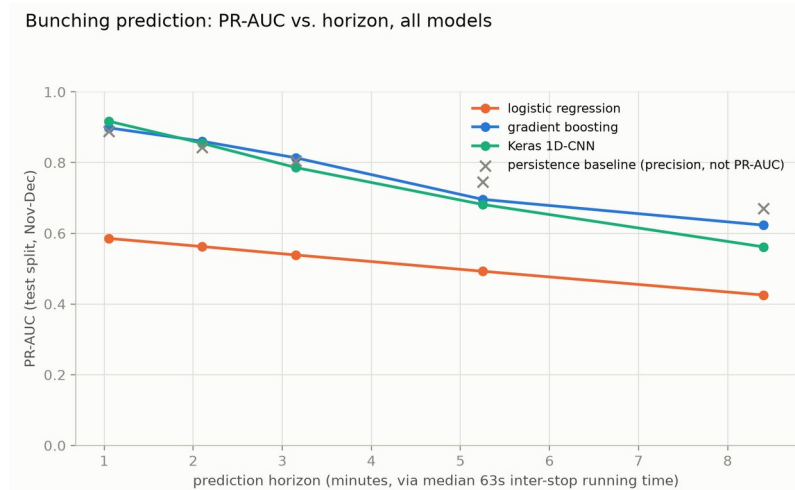


Figure 2. PR-AUC against prediction horizon for all learned models, with the persistence baseline marked as an operating point rather than a curve. The x-axis is lead time in minutes, which is the axis an operator cares about.

The baseline says the task is well posed. Persistence achieves 88.7 % precision and 84.1 % recall one stop ahead and decays smoothly to 66.0 % and 41.4 % at eight stops. Neither the near-perfect performance of a trivial label nor the immediate collapse of a noise-dominated one appears; instead there is a steady loss of predictability with distance, which is the signature of a real process. It also sets a demanding bar at short horizons: at $k = 1$ a learned model must add value on top of knowing whether the pair is bunched already.

Both nonlinear models clear it, and the margin widens with k . Gradient boosting runs from 0.898 PR-AUC at $k = 1$ to 0.623 at $k = 8$ against a 2.4 % base rate, and the Keras sequence model is essentially tied with it, winning narrowly at $k = 1$ (0.917) before falling behind from $k = 2$ onward. Where persistence loses more than half its recall between $k = 1$ and $k = 8$, the learned models retain a usable fraction of their discriminative power, which is the entire premise for building one: they can see a pair that is approaching bunching but has not arrived, and persistence by definition cannot.

Why the models rank the way they do. Logistic regression underperforms by a much wider margin than expected, 0.43 to 0.59 against 0.62 to 0.92, and that gap is itself a finding: the relationship is strongly nonlinear and interaction-heavy, and a linear model in these features cannot represent, for example, that a given delay means something different at an interchange in the afternoon peak than at a minor stop off-peak. The sequence model wins only at $k = 1$, where the raw recent shape of the headway trace is nearly sufficient, and loses at longer horizons where the tabular context features it does not see (line identity, route position, stop attributes, weather, leader state) carry more of the remaining signal. Calibration follows the same ordering (Brier 0.0128 for gradient boosting against 0.0208 for logistic regression at $k = 1$).

H3 is supported, with a sharper operational reading. The usable window is defined better by the last column of Table 2 than by PR-AUC. Holding precision at 80 %, gradient boosting catches 95 % of bunching events one stop ahead, 75 % three stops ahead, 47 % five stops ahead and only 20 % eight stops ahead. The practical answer to the research question is therefore that prediction is strong at one to three stops, roughly one to three minutes of lead time, degrades sharply beyond that, and is not usable at an eight-minute horizon at any acceptable false-alarm rate.

What the model actually uses. Permutation importance on the gradient booster at $k = 3$ is dominated by the current headway ratio (0.73), followed by arrival delay (0.127), segment speed (0.117) and the leader's arrival delay (0.091). The engineered closing-speed feature, expected in advance to dominate, contributes only 0.033. The most plausible explanation is collinearity rather than a broken feature: closing speed is algebraically a function of the headway ratio and its own lags, which are already in the model, and permutation importance measures marginal contribution given everything else. That explanation is plausible but unconfirmed; the clean test (removing the parent lags and re-measuring) was not run and is listed as future work rather than asserted.

6. Diagnostics and critical analysis

All diagnostics use the gradient boosting model at $k = 3$, the core horizon, scored once on the November to December test set. The model re-evaluated inside the diagnostic code returns 0.8136, matching the headline table, which confirms the breakdowns below describe the same model.

Experiment	PR-AUC	Reading
Full model, $k = 3$ (reference)	0.8136	baseline for the rows below
Delay features only	0.8188	best of the three arms
Delay and dwell features	0.8132	adding dwell does not help
Dwell features only	0.8034	weakest arm; H2 not supported
Leader features removed	0.7786	graceful, -4.3 % relative
Transfer model on its own 4 lines	0.7696	trained on half the data
Transfer model on 4 unseen lines	0.7726	no transfer penalty (RQ4)

Table 3. Ablation, robustness and transfer experiments. Each arm is a separately trained model on the same split and horizon.

H2 does not hold, and this is the most interesting result in the project. Delay-only features (0.8188) beat both dwell-only (0.8034) and the combined set (0.8132). Adding dwell on top of delay does not merely fail to help, it mildly hurts. This contradicts the mechanism the project was built on, in which dwell inflation is the causal driver, and it agrees with the independent permutation-importance evidence in Section 5, where every delay feature outranks every dwell feature. Two honest qualifications are needed before declaring the mechanism wrong. First, dwell is null for 16 % of rows, so the dwell arm carries more missingness noise than the delay arm. Second, and more seriously, the comparison is structurally unfair: the feature set contains a three-stop rolling delay sum but no rolling dwell equivalent, so what was tested is closer to “cumulative delay beats single-stop dwell” than to “delay beats dwell”. The result is reported as a rejection of H2 at this feature resolution, not as a refutation of the underlying dwell mechanism, and building the missing rolling dwell feature is the first item of future work.

The model's weakness is a line, not a condition. Per-line PR-AUC (Table 1) ranges from 0.922 on line 116 to 0.628 on line 474, a gap far larger than sampling noise at 112,923 test rows, and it is not explained by how much bunching a line has: lines 117 and 179 have higher base rates than 474 and score much better. The false-negative breakdown converges independently on the same answer: line 474 supplies 19.6 % of missed events but only 7.1 % of correctly caught ones. By contrast, performance is flat across peak and off-peak (0.811 against 0.817) even though the peak base rate is 3.3 times higher, and flat across weather conditions with adequate sample size (snow 0.814 at $n = 55,197$ against clear 0.817). The rain category scores 0.774, but at $n = 3,620$ and the lowest base rate in the table it should not be over-read. **This is the model-side test of H4**, and it supports the second half of the hypothesis cleanly: which features are predictive does not change with weather. The first half, that adverse weather raises the base rate, cannot be established here because the month-level signal is confounded by the summer timetable and by feed-reliability volatility (Section 3).

Failure mode and robustness. At a 0.5 threshold the model produces 30,759 true positives against 1,145 false negatives, so it misses very little, and the mean predicted probability among those misses is 0.197, not near zero. The model is not blind to them; it is assigning them real but sub-threshold probability, which means they are borderline cases a lower threshold would catch rather than a gap in the feature set. Removing the leader's features entirely, which simulates a single vehicle dropping off the real-time feed and taking with it the leader information of every trip behind it, costs 4.3 % relative

PR-AUC. That is a real cost but a graceful one: a deployed system would keep producing weaker predictions through a partial outage rather than going blind.

RQ4: transfer across lines is clean, within limits. A model trained only on lines 4, 541, 474 and 607 scores 0.7726 on the four lines it has never seen against 0.7696 on its own, a difference of -0.39 % that is well within noise. There is no meaningful transfer penalty, which matters practically: a new line can be onboarded without retraining from scratch. The limits are real, though. All eight lines are high-frequency trunk routes by construction of the selection rule, so this demonstrates generalisation within that regime, not to the low-frequency network where headways are long and the bunching dynamic is different. It is also a single four-against-four split rather than a leave-one-line-out design with a reported spread, which would have been the stronger test.

What the split choice is worth. Because the no-random-split rule and the validation set's mild October weather were both taken on principle, both were tested (Table 4). The whole-test-set column is not a like-for-like comparison, since the temporal split is scored on winter only while the alternatives are scored across the year; the controlled comparison re-scores each model on the November to December rows of its own test set. On that basis the random split does enjoy the expected leakage advantage, but it is 0.007 PR-AUC, or 0.9 % relative, small enough not to move any conclusion. All three splits land in a 0.79 to 0.82 band, so the headline result is not an artifact of the split. The weather-coverage concern, in contrast, is fully confirmed: the October validation set contains literally zero snowing or ice-risk rows, so it is blind to winter, not merely light on it. This matters specifically for the Keras model, whose early stopping was chosen against that set; the other models used validation for no decision. The weekly split fixes validation coverage but creates the mirror problem in its own test set (0.93 % snow against 4.2 % for the temporal split), so neither alternative replaces the temporal split, which remains the only one of the three that tests genuine forward prediction.

Split	Test PR-AUC (own test set)	PR-AUC on Nov-Dec rows only	Validation % snow / ice
Temporal (primary)	0.8136	0.8136	0.00 / 0.00
Random, trip level	0.7915	0.8207	3.88 / 3.84
Weekly round-robin	0.7876	0.7995	2.81 / 3.14

Table 4. Split-strategy comparison, same model and horizon. One seed per split, so these gaps are indicative rather than exact.

7. Practical deployment

What this supports is a controller decision aid at designated control points, not automated network-wide control. The evidence bounds the claim tightly. A one to three minute warning is enough for a control room to hold a following bus or flag a vehicle pair for attention, and the model is strong there (75 % of events caught at 80 % precision, three stops ahead). It is not enough for unattended automated triggering, which would want the eight-minute horizon where performance has collapsed to 20 % recall at the same precision.

Threshold choice is the deployment decision. At the default 0.5 threshold and $k = 3$ the model catches 96.4 % of events but issues 44,305 false alarms against 30,759 true ones, a precision of about 41 %. That is a system a control room would learn to ignore within a week. At the 80 % precision operating point, one alert in five is false and three quarters of events are still caught, which is a defensible starting point but is a reference, not a prescription: the correct threshold is an operational judgement about alert fatigue and should plausibly differ by line, since per-line performance varies by nearly 0.3 PR-AUC.

Where to pilot, and where not to. The central corridor cluster on line 4 (Garnisonen, Banérgatan, Skanstull, Värtavägen, Eriksdal, Musikhögskolan, Stadion, Östra station) is the strongest candidate: the highest bunching rates in the network, high observation volume, and a line the model predicts comparatively well. Line 474 should be explicitly excluded from any automated triggering in a first deployment, since it is the model's worst line on both per-line PR-AUC and its share of missed events. Operationally, scoring is not a constraint: batched inference costs 0.006 ms per row (6.1 ms for a single call, almost all of it Python overhead), so scoring every active vehicle on each real-time update is free at any credible fleet size.

Benefit, stated as a ceiling rather than a projection. Irregularity alone costs riders on these lines between 1.04 and 2.63 minutes of additional average wait beyond what the timetable implies (Table

1). That is the benefit ceiling if bunching were eliminated entirely; a real holding policy recovers some fraction of it, and this project does not estimate that fraction. Two reasons for caution are structural rather than incidental. Holding redistributes waiting time as well as reducing it, and without a demand model there is no way to convert an excess-wait reduction into realised passenger-minutes saved. More fundamentally, all performance here is measured on historical data from an uncontrolled system, while a deployed holding policy would change the very dynamics the model was trained on. A before-and-after pilot at the recommended control points is therefore not a nice-to-have but the only way to validate the benefit claim.

8. Conclusions, limitations and future work

Findings against the four hypotheses. H1 is supported in direction but not universally: headway CV rises along the route on two of eight lines and is flat on others, so route-position effects are line-specific. H2 is rejected at this feature resolution, with delay features outperforming dwell features and the combination beating neither, though the test is structurally unfair to dwell. H3 is supported and made precise: usable performance extends to roughly three stops, about three minutes, and collapses by eight. H4 is supported in its second and more interesting half, that weather does not change which features matter, while its first half is untestable here because of the summer-timetable confound. The core research question has a concrete answer: bunching can be predicted from operational data alone with one to three minutes of lead time at a defensible false-alarm rate, which makes it a controller decision aid rather than an automated control system.

Limitations. Eight high-frequency trunk lines out of roughly 536, selected on frequency and data completeness, so neither the descriptive findings nor the transfer result extends to the low-frequency network. Real-time completeness is volatile across the year, which weakens any seasonal reading. The validation set has no cold-weather examples, which affects the one model that used validation for a decision. Every ablation, transfer and split experiment is a single run at one seed and one horizon, so small gaps (the 0.007 leakage effect, the -0.39 % transfer difference) are indicative rather than precise. Service alerts are used only as a binary flag, with no text processing, so exogenous disruptions are separated only crudely from endogenous bunching.

Future work, in priority order. (1) Build a rolling multi-stop dwell feature and re-run the H2 ablation, which is the only way to settle whether the dwell mechanism is genuinely weaker or merely under-represented. (2) Repeat the line holdout as leave-one-line-out across all eight lines and report the spread rather than one number. (3) Isolate the closing-speed feature by removing its parent lags, to test the collinearity explanation directly. (4) Investigate line 474 specifically, since a 0.3 PR-AUC deficit concentrated on one line points to something structural (branching, a terminal pattern, or a corridor shared with other services) that the current feature set does not represent. (5) Extend the horizon study to lower-frequency lines, where the operational value of a longer warning is higher and the dynamics differ.

References

- Cats, O., Larijani, A.N., Koutsopoulos, H.N. and Burghout, W. (2011). Impacts of holding control strategies on transit performance: a bus simulation model analysis. *Transportation Research Record*, 2216, 51-58.
- Daganzo, C.F. (2009). A headway-based approach to eliminate bus bunching: systematic analysis and comparisons. *Transportation Research Part B*, 43(10), 913-921.
- Hersbach, H. et al. (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146(730), 1999-2049.
- Moreira-Matias, L., Mendes-Moreira, J., Gama, J. and Ferreira, M. (2015). On improving mass transit operations by using AVL-based systems: a survey. *IEEE Transactions on Intelligent Transportation Systems*, 16(4), 1636-1653.
- Newell, G.F. and Potts, R.B. (1964). Maintaining a bus schedule. *Proceedings of the 2nd Australian Road Research Board Conference*, 2, 388-393.
- Open-Meteo (2024). Historical Weather API, ERA5 archive. <https://open-meteo.com/en/docs/historical-weather-api>
- Pedregosa, F. et al. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Saito, T. and Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3), e0118432.